

Visione Artificiale

Il parte:

MACHINE LEARNING vs COMPUTER VISION

Raffaella Lanzarotti

References

- Deep Learning - Foundations and Concepts
 - Christopher M. Bishop, Hugh Bishop
 - <https://www.bishopbook.com/>
- Dive into Deep Learning
 - Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J Smola
 - <https://d2l.ai>

Where are we?

First part:

Image formation and Early vision

- Image formation
 - Geometric Camera Models
 - Color spaces
- Image Processing
 - Punctual and spatial processing
 - Feature Extraction
- Reconstruction
 - Camera calibration
 - Stereo Vision
 - Structure from Motion and RGB-d Cameras
 - Optical flow and Tracking

Second part:

Machine learning for CV

- **Linear Neural Network**
- Multi Layer Perceptron
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers
- Graph Neural Networks
- Generative Adversarial Networks

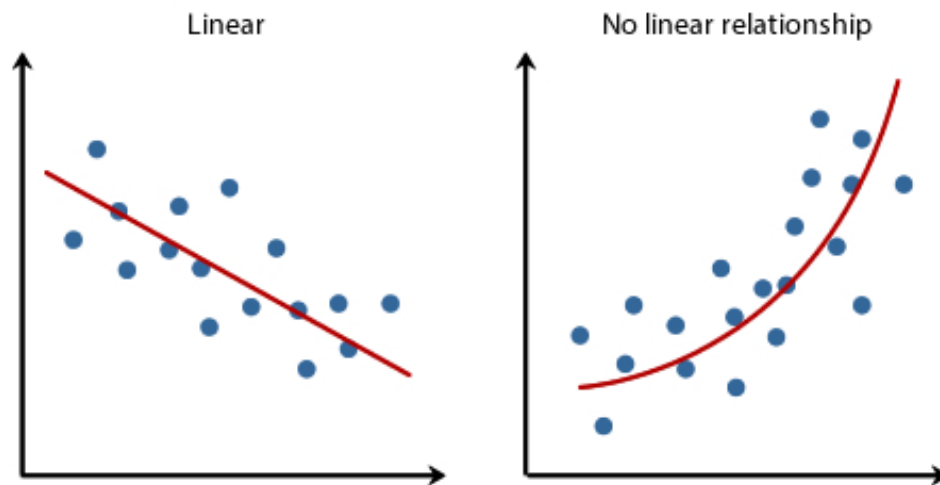
LINEAR NEURAL NETWORKS

Chapters 4,5 Deep Learning (Bishop)

Chapter 3, Dive into deep learning

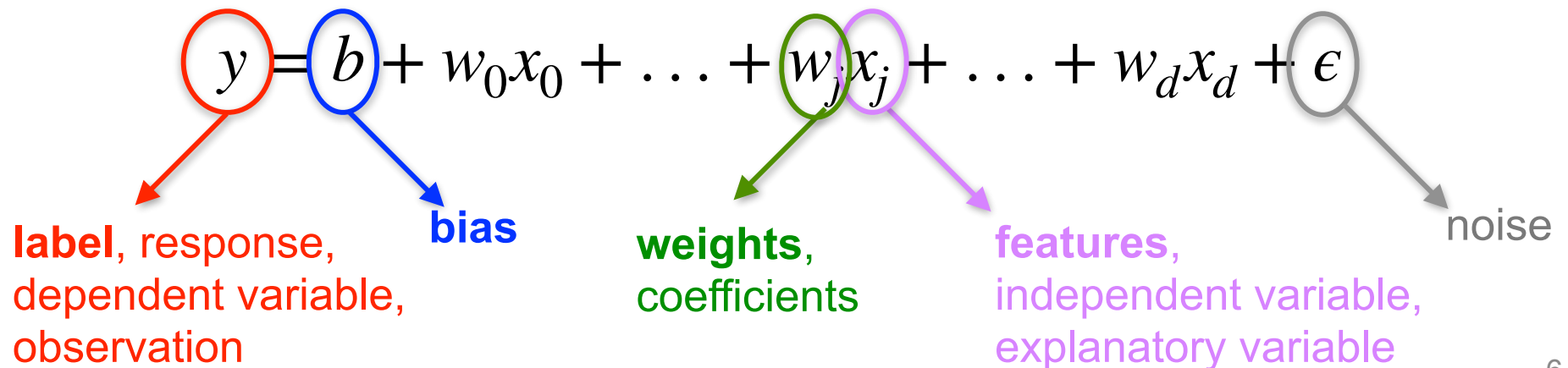
Regression

- **Regression**
methods for modeling the relationship between one or more **independent** variables and a **dependent** variable.
- In machine learning regression methods allow for **prediction**



Linear regression

- Assumptions:
 - **linear relationship** between independent variables $\mathbf{x}$ (input) and dependent variable y (output):
 - **noise** follows a **Gaussian** distribution



Example

- **GOAL**: estimate the prices (dollars) of houses based on their area (feet) and ages (years)
applying a linear model

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b$$

- We need a **Training set**:
a collection of **samples** characterized by
<area, age; price>


Example (cont.)

- **GOAL**: choose the best weights $\mathbf{w}$ and the bias b s.t., **on average**, the prediction $\hat{y}$ made by our model best fit the true prices y observed in the training set

The diagram shows the equation $\hat{y} = \mathbf{w}^T \mathbf{x} + b$ with several annotations:

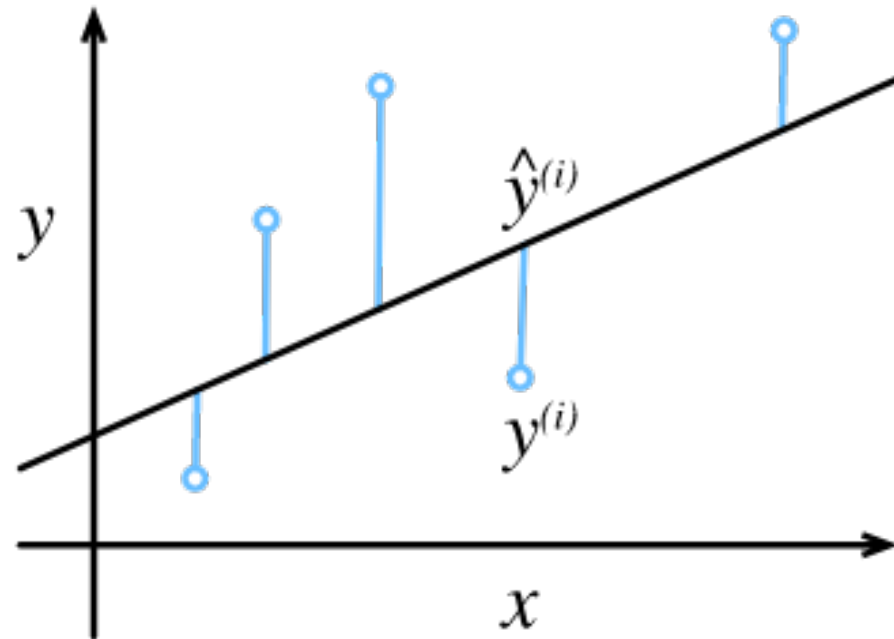
- A red circle around $\hat{y}$ with a red arrow pointing down to the text "label, a scalar in R ".
- A green circle around $\mathbf{w}$ with a green arrow pointing down and to the right to the text "weights vector in R^d ".
- A purple circle around $\mathbf{x}$ with a purple arrow pointing up and to the right to the text "feature vector in R^d , corresponding to a single data sample".

- referring to **multiple samples** $X \in R^{n \times d}$, we get multiple predictions $\hat{\mathbf{y}} \in R^n$

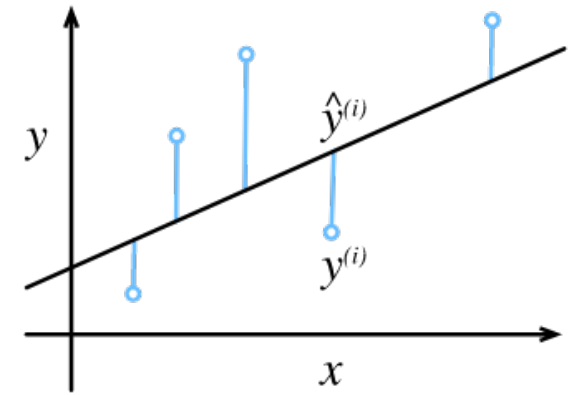
$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

Loss function

- **Loss function**: quantifies the distance between the *real* $y^{(i)}$ and the *predicted* $\hat{y}^{(i)}$ value of the target



Loss function



Squared loss:

- On a single example i :

$$\ell^{(i)}(\mathbf{w}, b) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$

- On the whole dataset:
(average the losses obtained on the single data samples):

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \ell^{(i)}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

Motivation of the squared loss function

- Hypothesis: observations (y) are the noisy linear combination of features ($\mathbf{x}$)

$$y = \mathbf{w}^T \mathbf{x} + b + \epsilon, \quad \epsilon \approx N(0, \sigma^2)$$

- Likelihood of seeing a y for a given $\mathbf{x}$:

$$P(y | \mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{1}{2\sigma^2} (y - \mathbf{w}^T \mathbf{x} - b)^2 \right)$$

the residuals ($y - \mathbf{w}^T \mathbf{x} - b$) follow a normal distribution, which leads to a Gaussian likelihood function.

- Maximize likelihood on the entire dataset. $P(\mathbf{y} | \mathbf{X}) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)})$
- Equivalent to maximize its log, or minimize -log:

$$-\log P(\mathbf{y} | \mathbf{X}) = \sum_{i=1}^n \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} - b)^2$$

Independent of the parameters

Find the best model by training

- GOAL:

find the parameters $(\mathbf{w}^*, b^*)$ that minimize the total Loss:

$$\mathbf{w}^*, b^* = \operatorname{argmin}_{\mathbf{w}, b} L(\mathbf{w}, b)$$

- ➔ Optimization should ideally satisfy two conditions to be considered technically sound:
- existence
 - uniqueness

Closed-form Solution (for linear model)

- Add bias into weights by $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}]$ $\mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

- The **Loss** is convex quadratic polynomial function

$$L(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} \left\| \mathbf{y} - \mathbf{X}\mathbf{w} \right\|^2$$

- **global minimizers** via **derivatives** (w.r.t. $\mathbf{w}$) equated to 0:

$$0 = \frac{\partial}{\partial \mathbf{w}} L(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

$$\Leftrightarrow (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X} = 0$$

- when $\mathbf{X}$ has full-rank, Normal Equations gives:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Find the best model by training

- GOAL: find the parameters $(\mathbf{w}^*, b^*)$ that minimize the total Loss:

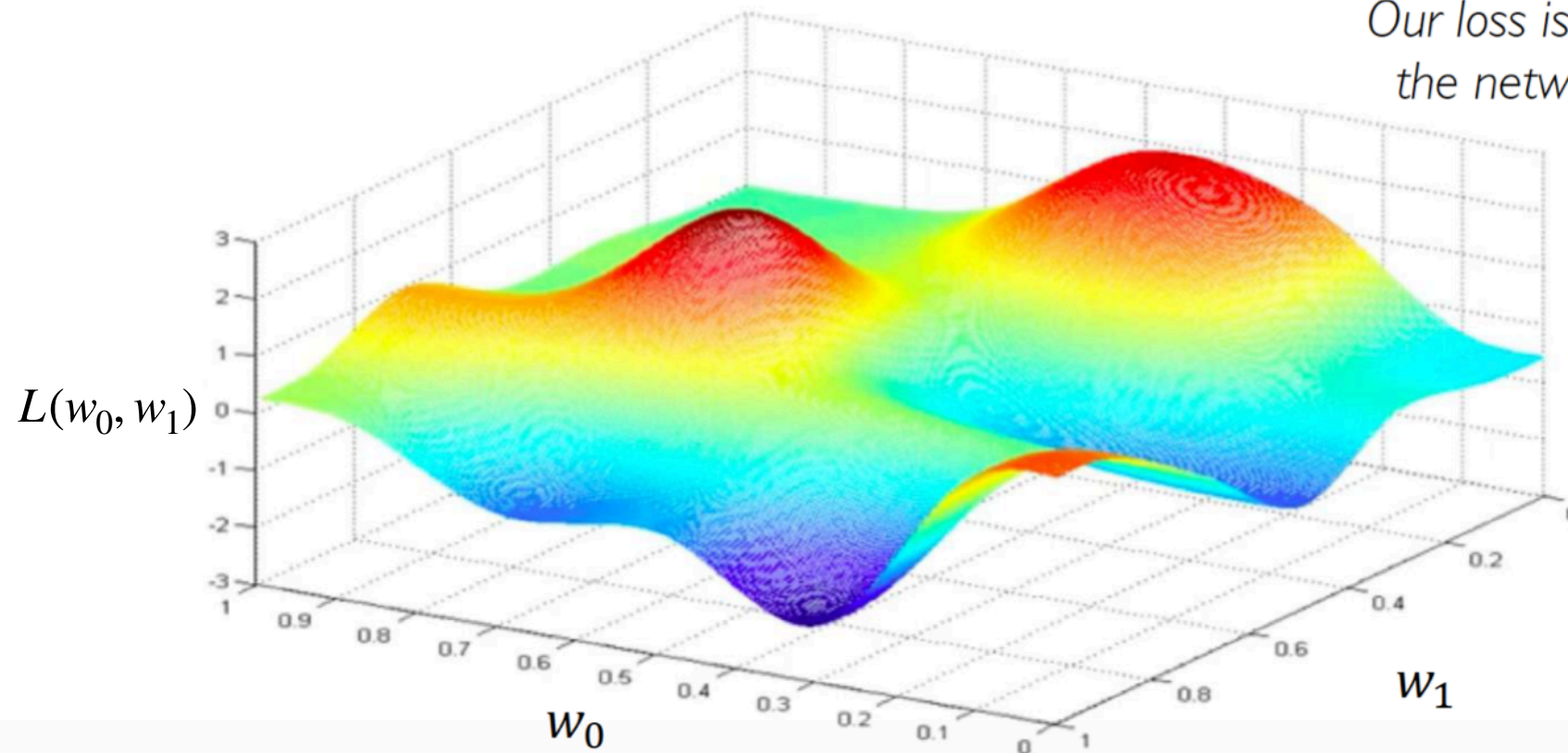
$$\mathbf{w}^*, b^* = \operatorname{argmin}_{\mathbf{w}, b} L(\mathbf{w}, b)$$

- Alternative: **iterative** method yielding sequence $\{(w_i^*, b_i^*)\}_{i=0}^{\infty}$ converging to a solution
- Machine learning community says “**by training**”:
 - **iterative optimization** algorithm for updating the parameters toward some direction that incrementally lowers L
 - e.g.: **Gradient Descent** method, if applicable

Find the best model by training

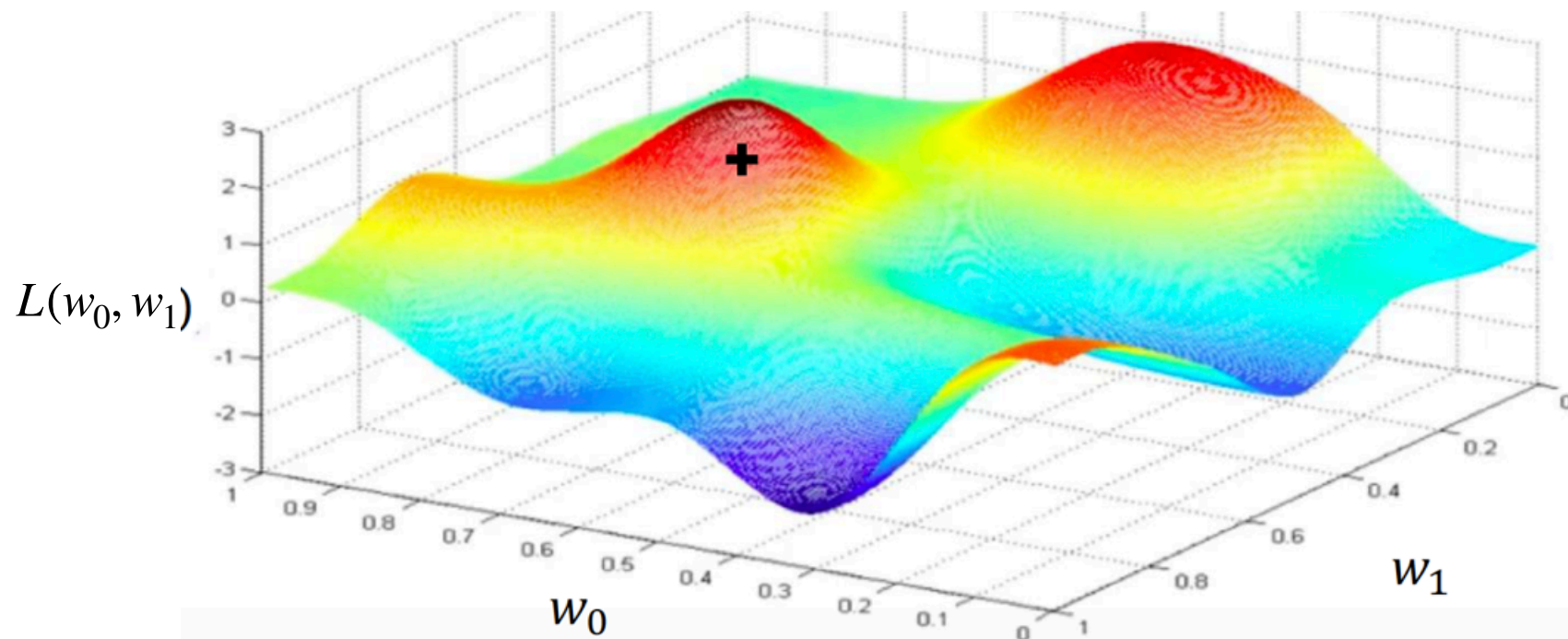
$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} L(\mathbf{w})$$

Remember:
*Our loss is a function of
the network weights!*



Find the best model by training

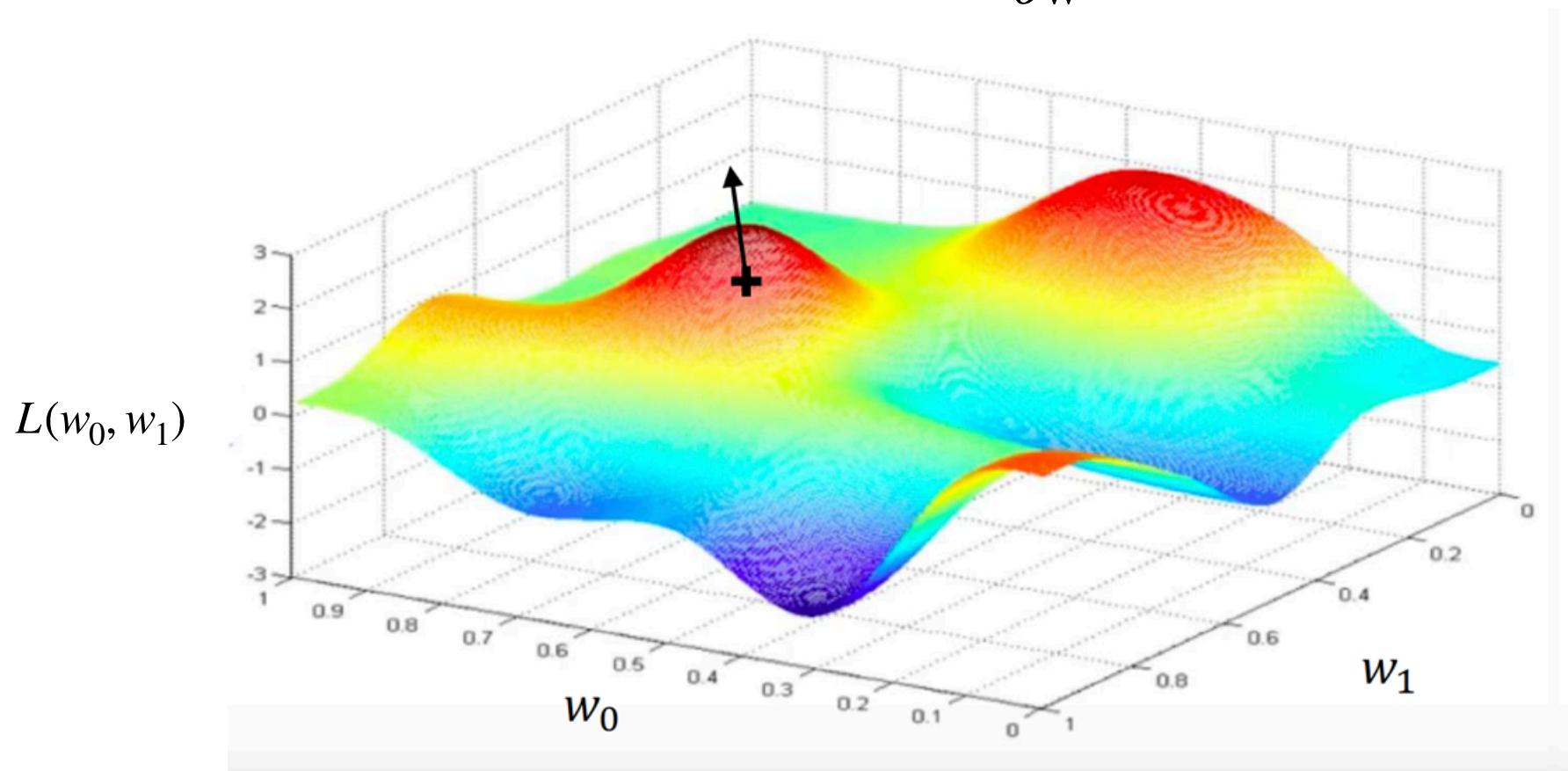
Loss optimization through **gradient descent**



□

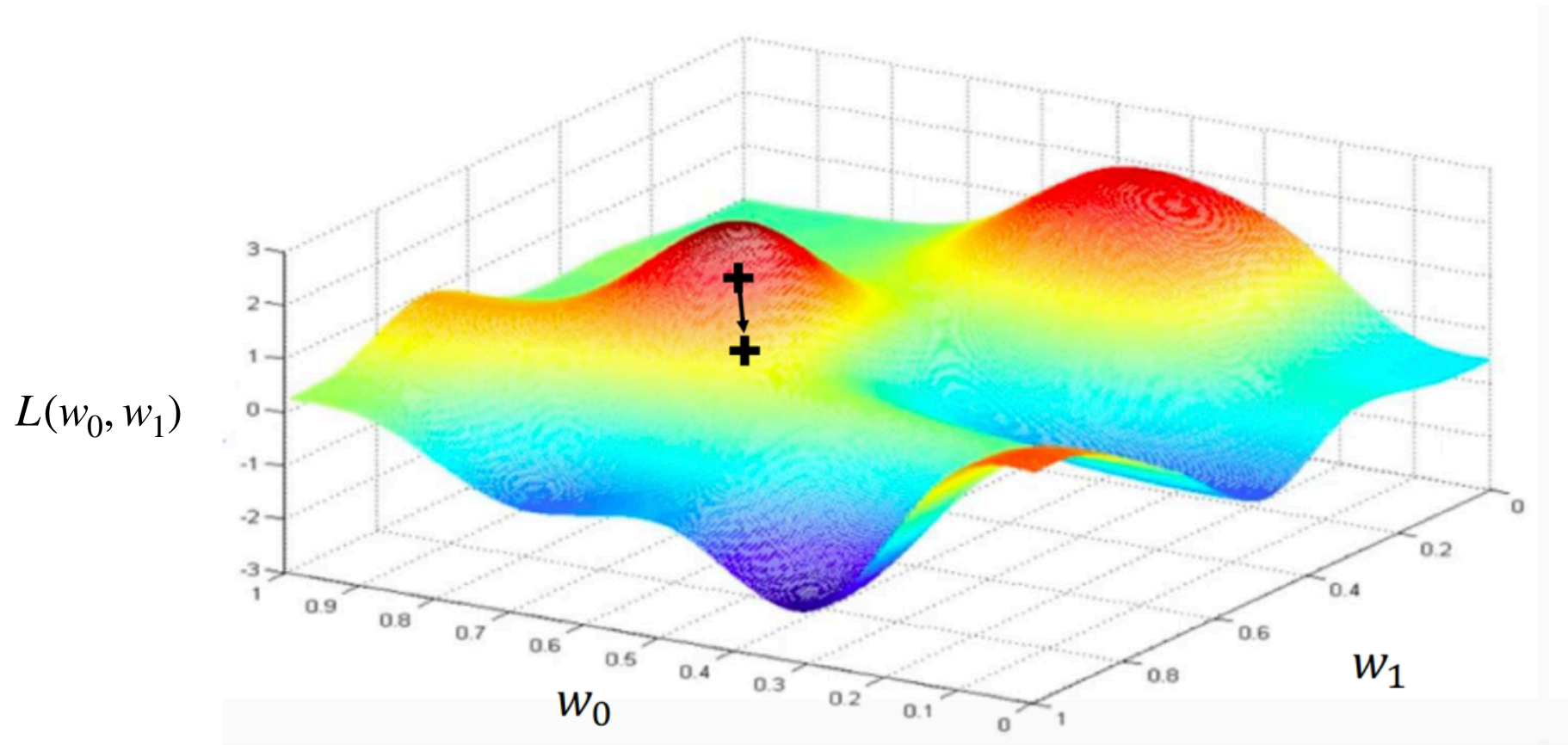
Find the best model by training

Compute the gradient $\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}$



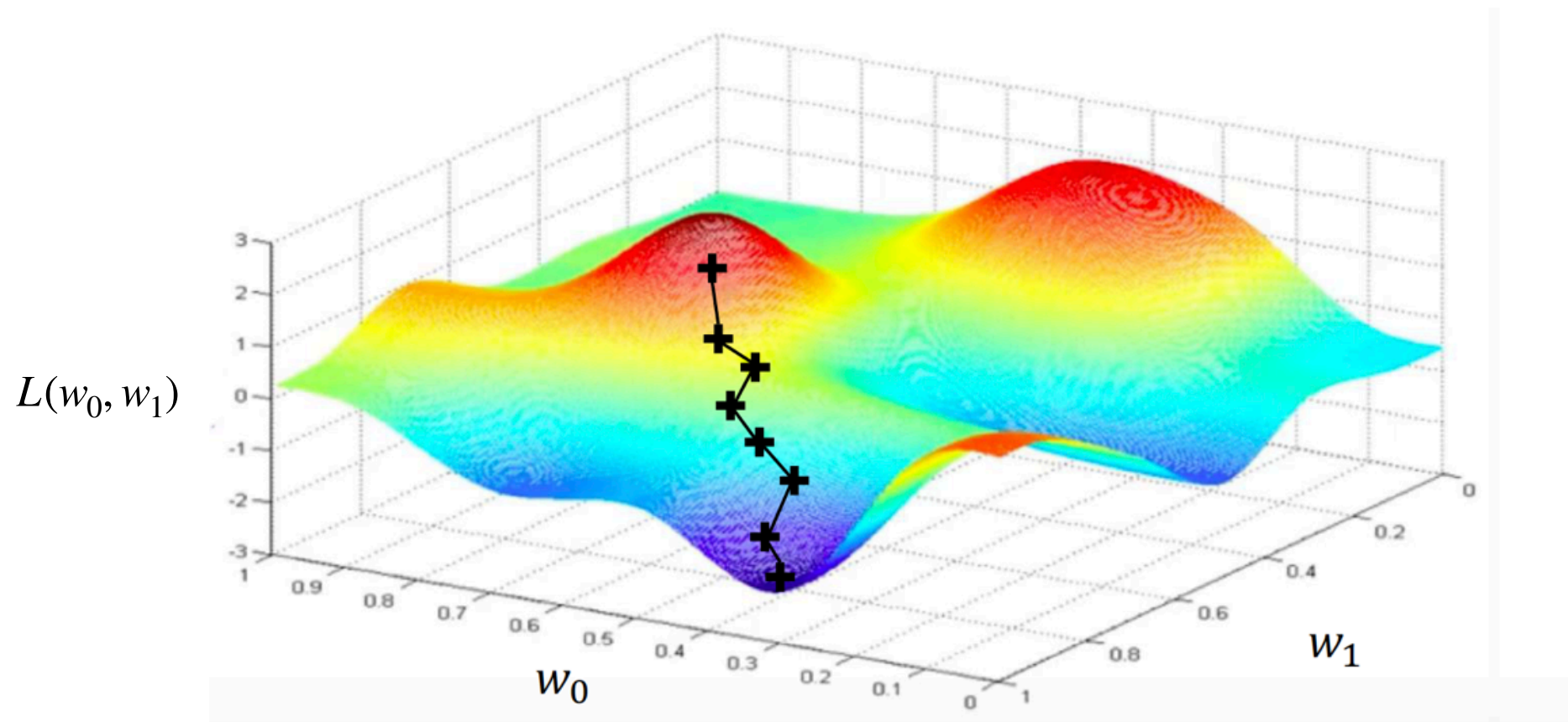
Find the best model by training

Take small step in opposite direction of gradient



Find the best model by training

Repeat until convergence



Gradient descent

Algorithm

1. Initialize the weights randomly $\approx N(0, \sigma^2)$
2. Loop until convergence
 - 2.1. Compute gradient $\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}$
 - 2.2. Update weights $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}$
3. Return weights

Find the best model by training

- GOAL: find the parameters $(\mathbf{w}^*, b^*)$ that minimize the total Loss:

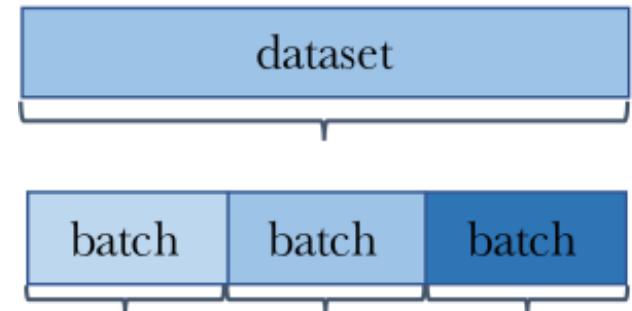
$$\mathbf{w}^*, b^* = \operatorname{argmin}_{\mathbf{w}, b} L(\mathbf{w}, b)$$

- Alternative: iterative method yielding sequence $\{(w_i^*, b_i^*)\}_{i=0}^{\infty}$ converging to a solution
- Machine learning community says “by training”:
 - iterative optimization algorithm for updating the parameters toward some direction that incrementally lowers L
 - e.g.: Gradient Descent method, if applicable
 - **drawback**: computing gradient of L over the whole training set (X, y) to produce a single update is **too expensive**

Find the best model by training

Minibatch Stochastic Gradient Descent (SGD):

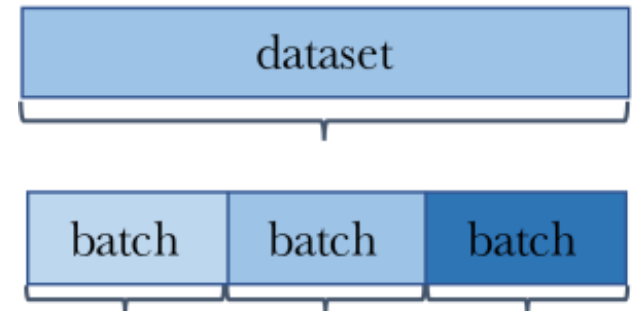
- initialize the weights randomly
- Loop until convergence:
 1. randomly select a **minibatch** β
(a chosen subset of training examples)



Find the best model by training

Minibatch Stochastic Gradient Descent (SGD):

- initialize the weights randomly
- Loop until convergence:



1. randomly select a **minibatch** β
(a chosen subset of training examples)
2. compute the gradient of L restricted on β w.r.t. the model parameters, $\partial_{(\mathbf{w},b)} \ell^{(i)}(\mathbf{w}, b)$
3. update the parameters:

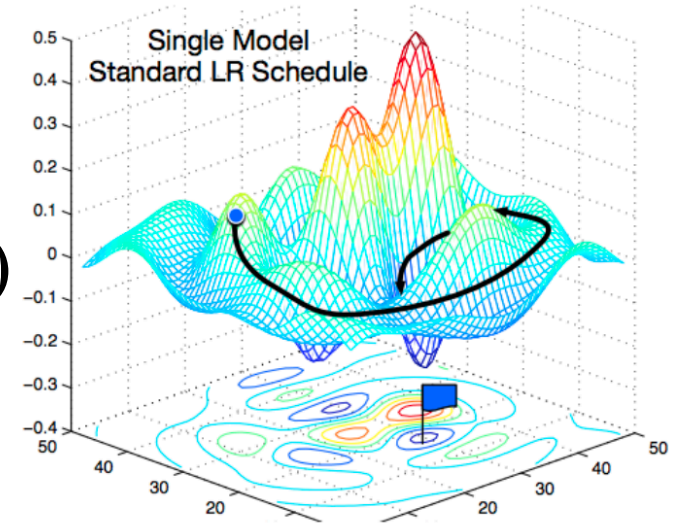
$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\beta|} \sum_{i \in \beta} \partial_{(\mathbf{w},b)} \ell^{(i)}(\mathbf{w}, b)$$

Learning rate and batch size

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\beta|} \sum_{i \in \beta} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b)$$

learning rate

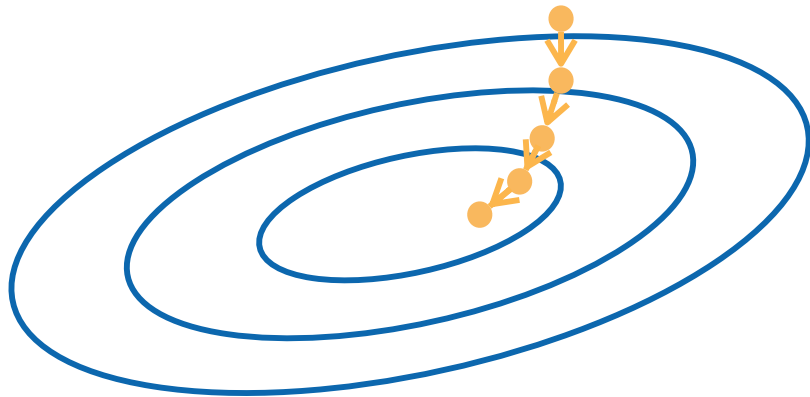
batch size



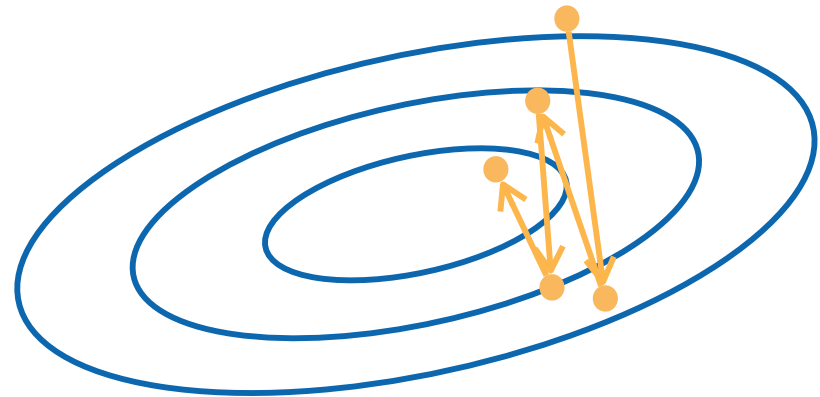
- η and $|\beta|$: **hyperparameters**
- We would like iterations to converge to a minimizer point.
- How to choose η and $|\beta|$?
 - manually pre-specified and eventually subjected to **tuning** (in a validation procedure)
 - η : scheduled by specific **optimization algorithms**

Choose a Learning Rate η

Not too small



Not too big



Choose a Batch Size $|\beta|$

- Not too small

Workload is too small,
hard to fully utilize
computation resources

- Not too big

Memory issue
Waste computation,
e.g. when all x_i are
identical

Some terminology

- Why "**Stochastic**" Gradient descent? due to the random choice of β
- **Batch size**: number of training examples in a mini-batch
- **Iteration**: is the number of batches needed to complete one epoch
- **Epoch (in SGD)**: a full pass through the entire dataset, by as many iteration as # of mini-batches
- Ex. For a dataset of 10000 sample with mini-batch size 1000, 10 iterations will complete 1 epoch

Linear Model, sum up

- **Linear model**

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

- d -dimensional **inputs** $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$

- **Parameters** to learn: $\mathbf{w} = [w_1, w_2, \dots, w_d]^T$ and b

- **Loss function** (to measure goodness of fit) $l(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$,

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Learn by mini-batch SGD**

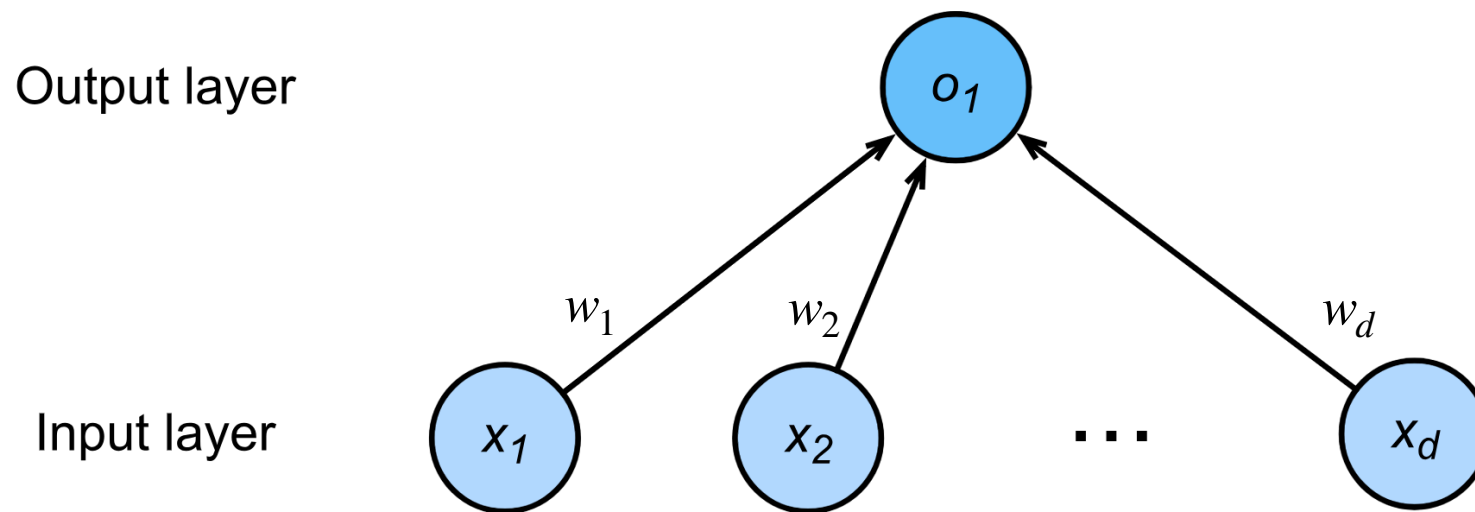
- Choose a starting point $\mathbf{w}_0, b_0$
- Repeat:
 - Compute gradient
 - Update parameters

- **Test phase:**

- apply the learnt model to new data $\mathbf{x}$ to predict the corresponding target y

Linear Model as a Single-layer Neural Network

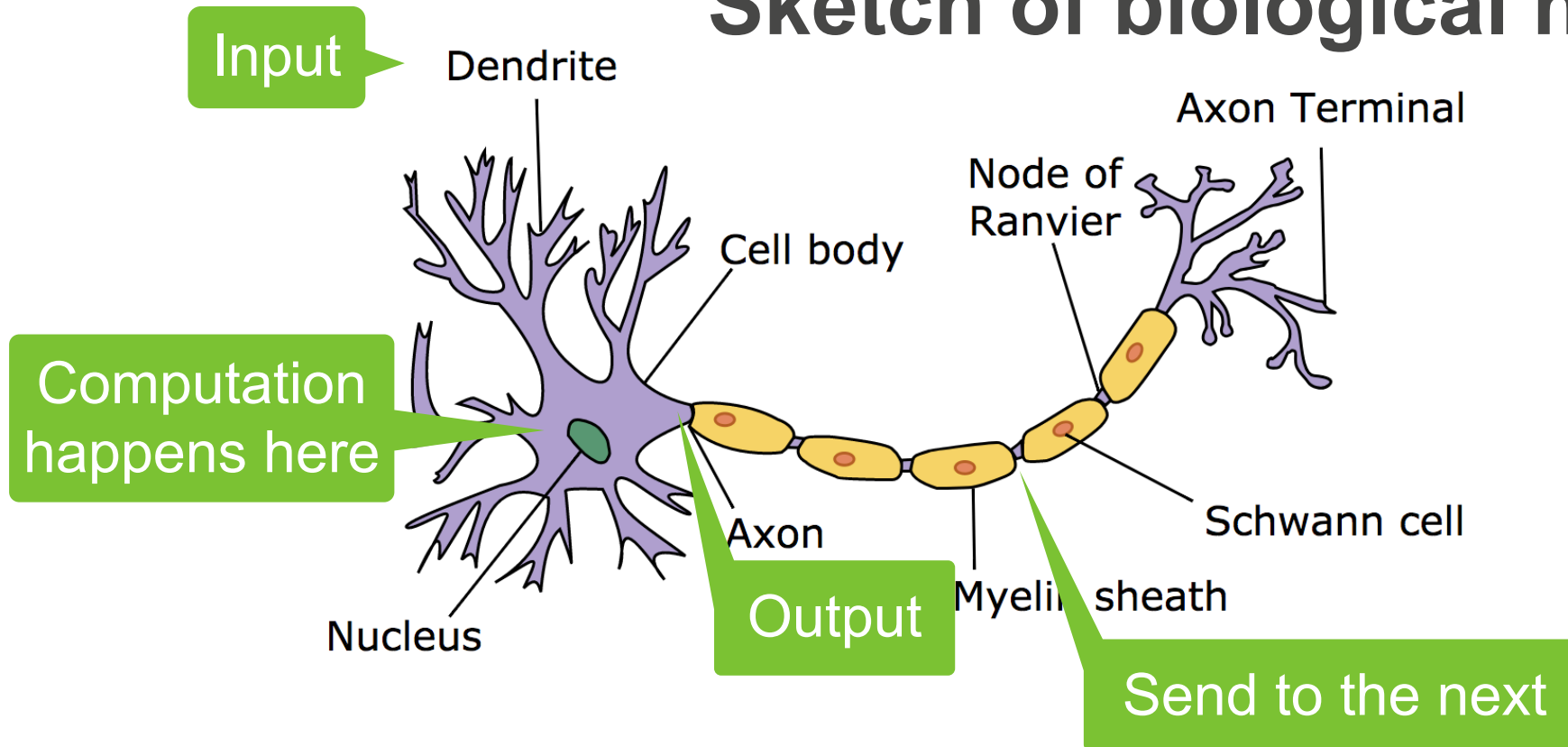
- Linear regression as a NN with one layer (input is not counted).
- Input are d -dimensional features
- Output is mono-dimensional



[By stacking multiple layers, and wiring one layer's output to subsequent layer's input, we get deep neural networks]

NNs arise from Neuroscience discoveries

Sketch of biological neuron



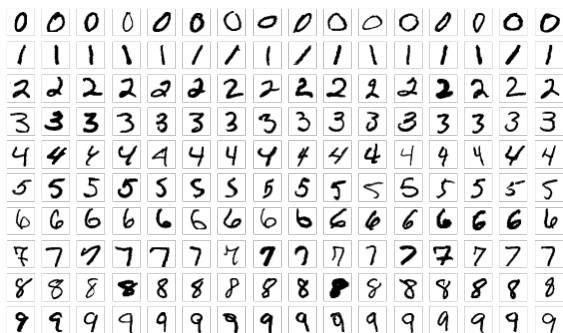
- The information arriving from neurons via **dendrites** is weighted by **synaptic weights** determining the effect of the input (activation/inhibition)
- the input arriving from several neurons are aggregated (weighted sum) in the **nucleus** and sent to other neurons after applying a non linear function

Regression vs Classification

- **Regression** estimates a continuous value
- **Classification** predicts a discrete category

MNIST:

classify hand-written digits
(10 classes)



ImageNet:

classify nature objects
(1000 classes)



Softmax regression

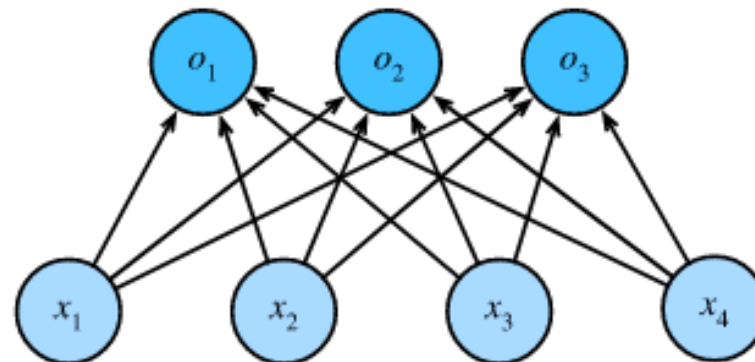
To solve **classification** problems
(eg: classify an image according to its content)

- Example:
 - 4 pixels as input and 3 categories as output (cat, dog, pig)
 - **label** coded with the **one-hot-encoding**:
 $y \in \{(1,0,0), (0,1,0), (0,0,1)\}$
 - estimate $\mathbf{o}$, applying:
 $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $|\mathbf{W}| = 3 \times 4, |\mathbf{b}| = 3$

Called
fully-connected
layer

Output layer

Input layer



Softmax operation

- We would like an output $\hat{y}_j$ s.t. it could be interpreted as “the probability the input belongs to class j ”
 - eg: $\hat{y}_1 = 0.1$, $\hat{y}_2 = 0.8$, $\hat{y}_3 = 0.1$
 - classification: $\operatorname{argmax}_j \{\hat{y}_j\}$
 - Unfortunately: $\hat{y}_j \neq \mathbf{o}_j$, since:
 1. $\mathbf{o}_j$ can be negative
 2. $\sum_j \mathbf{o}_j \neq 1$
- So, $\mathbf{o}_j$ cannot be interpreted as a probability

Softmax function

- **Idea**: introduce an activation function for transforming logits into probabilities (in a differentiable manner):

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}), \quad \text{where } \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}$$

- *Properties*:

$$\begin{aligned} & \bullet \sum_j \hat{y}_j = 1 & \bullet 0 \leq \hat{y}_j \leq 1 & \bullet \operatorname{argmax}_j \{\hat{y}_j\} = \operatorname{argmax}_j \{\hat{o}_j\} \end{aligned}$$

Loss function

- The **Loss** quantifies how “poor” the prediction is
- Common choices:

- For **regression** problems **SQUARED LOSS**:

$$l = (y_i - \hat{y}_i)^2$$

- for **binary classification** problems **BINARY CROSS ENTROPY LOSS**:

$$l = -\frac{1}{2} \sum_{j=1}^2 y_j \cdot \log(\hat{y}_j) + (1 - y_j) \cdot \log(1 - \hat{y}_j)$$

- for **multiclass classification** problems: **CROSS ENTROPY LOSS**:

$$l = - \sum_{j=1}^q y_j \cdot \log(\hat{y}_j)$$

Cross-entropy loss and derivatives

- Let plugin the softmax definition into the cross-entropy loss:

$$\begin{aligned}l(\mathbf{y}, \hat{\mathbf{y}}) &= - \sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \\&= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \\&= \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j.\end{aligned}$$

Difference between true label and estimated probability

- The derivative wrt any logic $\mathbf{o}_j$ is:

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j = \text{softmax}(\mathbf{o})_j - y_j.$$

Linear Model vs Softmax Regression

Problem	Single value regression	k-class classification
Model	$\langle \mathbf{w}, \mathbf{x} \rangle + b$ $\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}$	$\text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$ $\mathbf{W} \in \mathbb{R}^{k \times n}, \mathbf{b} \in \mathbb{R}^k$
Loss	Squared loss	Cross-entropy loss

Notebooks

- Concise Implementation of Linear regression
- Image classification dataset
- Concise implementation of Softmax Regression